花卉识别作业报告（任务三 + 任务四）

数据集：flower 数据集，共 5 类花（daisy / dandelion / rose / sunflower / tulip），原始 jpg 共 4320 张，官方已分为 train 和 test 两个文件夹。

文件说明：

- `01_数据预处理.py` 任务三的源码，运行后生成处理好的数据集 `flower_data/`
- `02_模型与训练.py` 任务四的源码，包含网络结构、训练、画曲线
- `预处理日志.txt` / `训练日志.txt` 两个脚本的实际运行输出
- `训练曲线.png` 准确率和损失随训练进度的变化曲线

任务三：数据预处理

1. 清理损坏图片

方法：用 PIL 遍历数据集里的每一张图，先 `Image.verify()` 检查文件头，再重新打开执行 `load()` 真正解码一遍像素，两步都通过才算好图；任何一步异常就判定为损坏，移动到 `bad_images` 文件夹（不直接删，防止误判，但是我并没有上传到这次作业里面）。

踩坑记录：第一版只检查了 `.jpg`，训练时直接报 `UnidentifiedImageError`，排查才发现数据集里混了 10 个 `.png` 文件,发现全是无法打开的 0~28 字节的坏文件。所以检查范围改成了 jpg + png，顺便把有效的非 jpg 图片统一转存为 jpg，保证整个数据集格式一致。

结果：共扫描 4330 张图，查出 13 张损坏（3 张坏 jpg + 10 个假 png），全部移入 bad_images。

2. 划分数据集

官方只给了 train 和 test。训练过程中需要一个验证集来观察模型有没有过拟合、该不该早停，所以把 train 按 8:2 再切成训练集和验证集。切之前按类别分别打乱（`random.seed(2024)` 固定随机种子，保证每次运行划分结果一样，实验可复现），test 保持原样不动。

结果：train 2763 张 / val 694 张 / test 860 张。

3. 数据增强

用 torchvision 的 transforms 实现，只对训练集做随机增强，相当于每个 epoch 模型看到的图都不完全一样，等价于扩充了样本多样性：

变换	作用
Resize(256) + RandomCrop(224)	先缩放再随机裁剪，每次裁的位置不同
RandomHorizontalFlip	随机左右翻转（花朵没有左右方向之分，翻转不改变语义）
RandomRotation(15)	随机旋转 ±15 度，模拟拍摄角度差异
ColorJitter(0.2, 0.2, 0.2)	随机微调亮度/对比度/饱和度，模拟光照差异

验证集和测试集不做随机增强，只做 Resize + CenterCrop，保证评估指标稳定可比。

4. 调整图片尺寸

所有图片统一处理成 224×224，正好是网络结构图标注的输入尺寸。ToTensor 之后还做了 Normalize（减均值除标准差，用的是 ImageNet 的统计值），让输入分布更平稳、训练更稳定。

任务四：网络设计与训练

1. 网络结构

按作业图逐层搭建（就是 AlexNet 的结构）：

层	配置	输出尺寸
输入	3 通道 224×224	3×224×224
C1	Conv 11×11, stride=4, padding=2, 96 通道 + ReLU + MaxPool	96×55×55 → 96×27×27
C2	Conv 5×5, padding=2, 256 通道 + ReLU + MaxPool	256×27×27 → 256×13×13
C3	Conv 3×3, padding=1, 384 通道 + ReLU	384×13×13
C4	Conv 3×3, padding=1, 384 通道 + ReLU	384×13×13
C5	Conv 3×3, padding=1, 256 通道 + ReLU + MaxPool	256×13×13 → 256×6×6
FC6	Linear 9216→4096 + ReLU + Dropout(0.5)	4096
FC7	Linear 4096→4096 + ReLU + Dropout(0.5)	4096
FC8	Linear 4096→类别数	5

一点说明：结构图上 FC8 画的是输出 18 类，但本数据集只有 5 种花，所以最后一层输出改成 5。C1 用 padding=2 是为了对上图里 55×55 的标注：(224+2×2−11)/4+1 = 55，三处 MaxPool 参数相同（3×3, stride=2），代码里共用一个 `self.pool`。每层尺寸都和图上一一对应，forward 里逐层核验过。

2. 训练设置

- 损失函数：交叉熵 CrossEntropyLoss（多分类标配）
- 优化器：Adam，学习率 1e-4（Adam对学习率不敏感，所以就直接用他省调参精力）
- batch_size=32，epoch=10
- 设备：CPU（本机无独立显卡）

3. 训练结果

epoch	训练准确率	验证准确率
1	30.84%	48.99%
3	50.67%	55.91%
5	61.89%	64.41%
8	67.90%	66.14%
10	70.54%	70.61%

最终在测试集上的准确率：**74.07%**（损失 0.7293）。

曲线见 `训练曲线.png`：验证准确率一路跟着训练准确率往上走，两条线基本贴合，没有出现明显的过拟合（因为加上了Dropout）；到第 10 个 epoch 时曲线还没有完全走平，说明继续加 epoch 或调大学习率应该还能再涨一些。

4. 复现方式

```
python 01_数据预处理.py   # 先跑预处理，生成 flower_data
python 02_模型与训练.py   # 再训练，输出日志、曲线和模型权重
```

依赖：pytorch、torchvision、Pillow、matplotlib。